AI with Rav

Learn AI the way you actually think.

DAY 19 of 30

MACHINE LEARNING

Feature Engineering — the secret weapon

WHAT YOU'LL LEARN TODAY

- Why prep beats a fancy model
- Splitting one column into many
- Turning words into numbers
- Scaling — the same ruler
- Combining columns into a smarter clue

Feature Engineering — the secret weapon

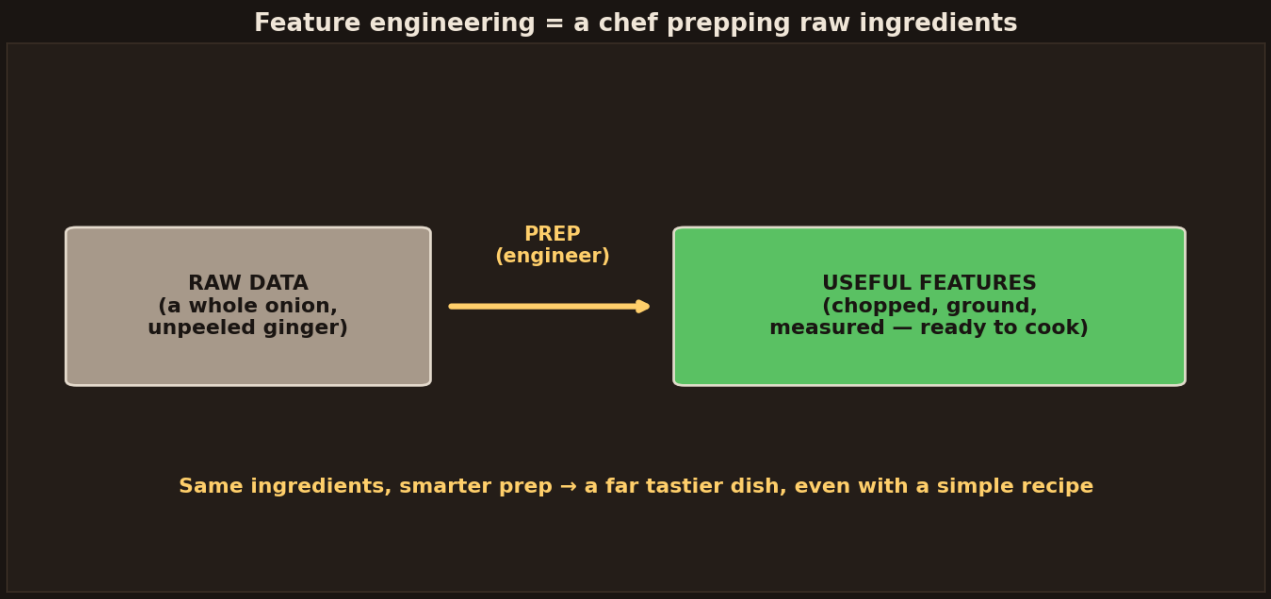
> In One Line: A great chef doesn't just cook — they PREP. They chop the onion, grind the ginger, measure the spices. Feature engineering is that prep for data: the same raw ingredients, prepared smartly, make even a simple model taste amazing.

Start here: the skill that beats a fancy model

There's an open secret among ML engineers: **better features usually beat a fancier model.** A simple model with well-prepared features will often crush a complex model fed raw, messy data. This one skill — **feature engineering** — is what separates good engineers from great ones.

Think of a chef. You can hand two chefs the exact same vegetables. The average chef throws them in a pot whole. The great chef **preps** them — chops, grinds, measures — and gets a far tastier dish from the *same* ingredients. Feature engineering is that prep.

Raw data is like raw vegetables

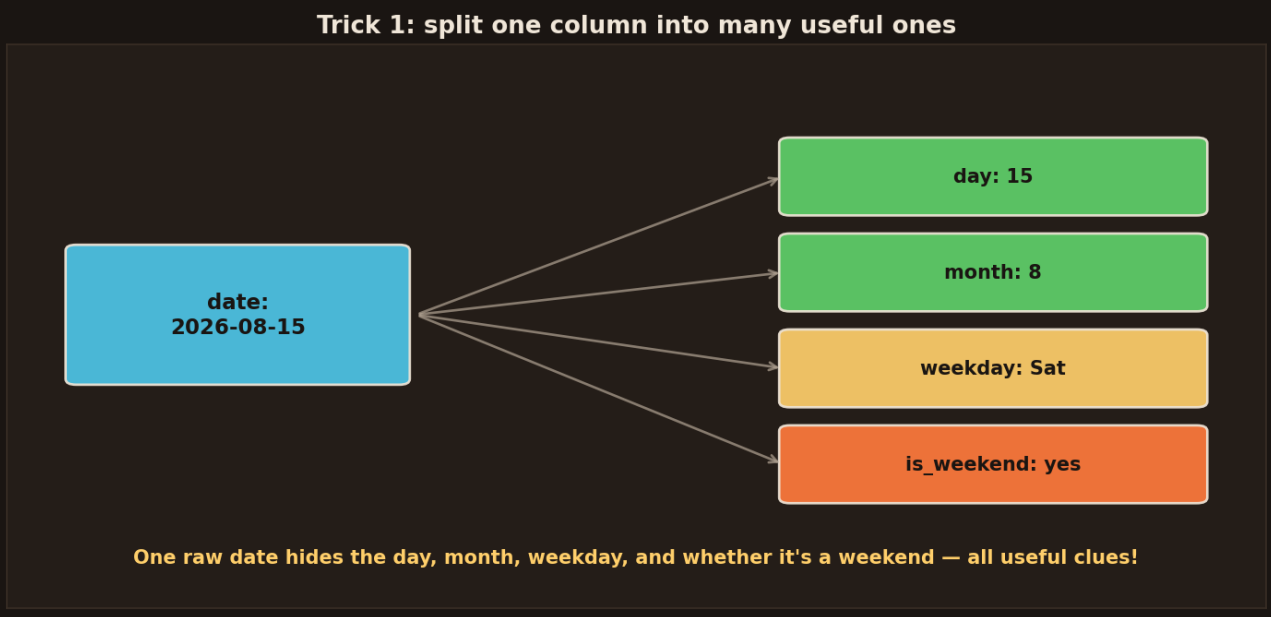


Raw data is like a whole onion and unpeeled ginger — not ready to cook. Feature engineering is the PREP that turns it into useful, ready-to-use features. Same ingredients, smarter prep, a far better dish.

- **Raw data** = a whole onion, unpeeled ginger, a raw date. Not ready.
- **Prep (feature engineering)** = chop, grind, measure — turn it into a form the model can actually use.
- Same ingredients + smarter prep → a **far tastier dish**, even with a simple recipe (model).

> The model is only the "cooking" step. If you feed it raw, unprepped data, even the best recipe fails. But prep the ingredients well, and a simple model shines. Let's learn the chef's four prep tricks.

Trick 1: split one column into many



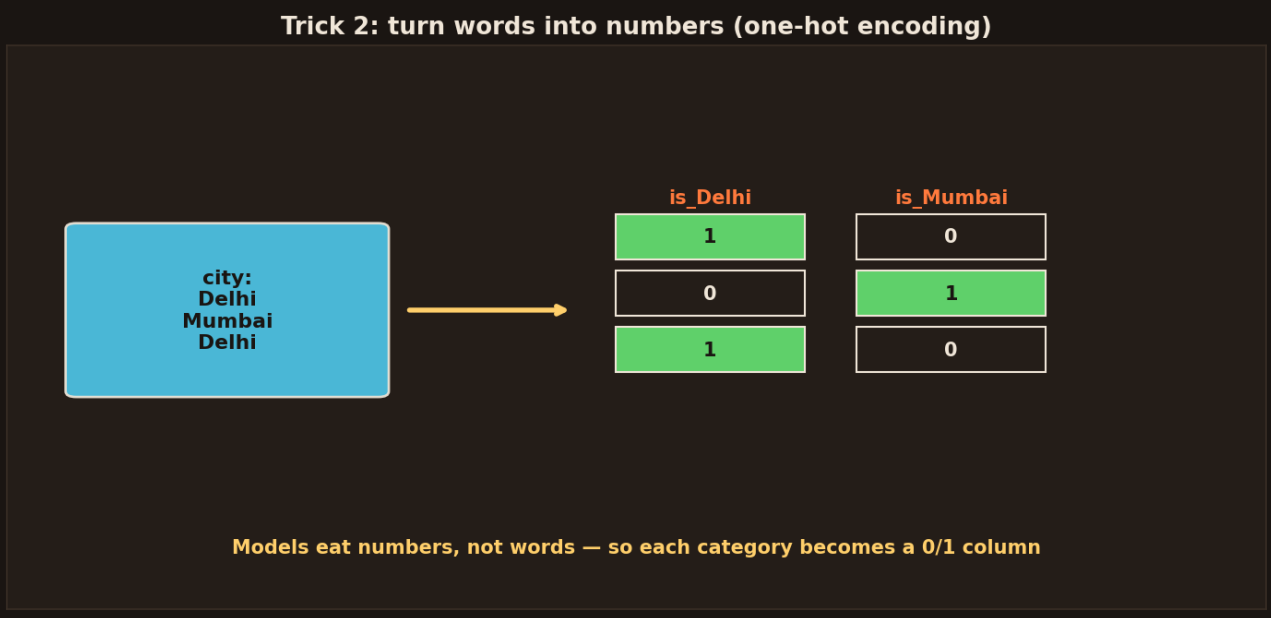
A single "date" column secretly holds a day, a month, a weekday, and whether it's a weekend. Splitting it into those separate columns hands the model four useful clues instead of one confusing blob.

Raw columns often **hide** several clues inside one. A date like `2026-08-15` is useless to a model as-is — but split it open:

- **day** = 15, **month** = 8 → catches monthly or seasonal patterns.
- **weekday** = Saturday → catches "weekends are different."
- **is_weekend** = yes → a clean yes/no clue.

> One raw date became **four useful features.** Sales spike on weekends? Now the model can see it. This is the most common prep trick: take a rich column (date, address, name) and **split it into the useful pieces hiding inside.**

Trick 2: turn words into numbers



Models can't read words. A "city" column of Delhi/Mumbai becomes two number columns — is_Delhi and is_Mumbai — each holding a 1 or 0. This is called one-hot encoding.

Here's a hard rule: **models eat numbers, not words.** A column like `city = Delhi / Mumbai` means nothing to the maths. So we turn each category into a **0/1 column** — a trick called **one-hot encoding:**

- Make a column **is_Delhi** and a column **is_Mumbai.**
- For a Delhi row → is_Delhi = 1, is_Mumbai = 0.
- For a Mumbai row → is_Delhi = 0, is_Mumbai = 1.

> Now the words are numbers the model can do maths on. One-hot encoding is how you feed *any* category — city, colour, product type — into a model. (Careful: hundreds of categories make hundreds of columns; there are smarter encodings for that, but one-hot is the everyday workhorse.)

Trick 3: scaling — put everything on one ruler

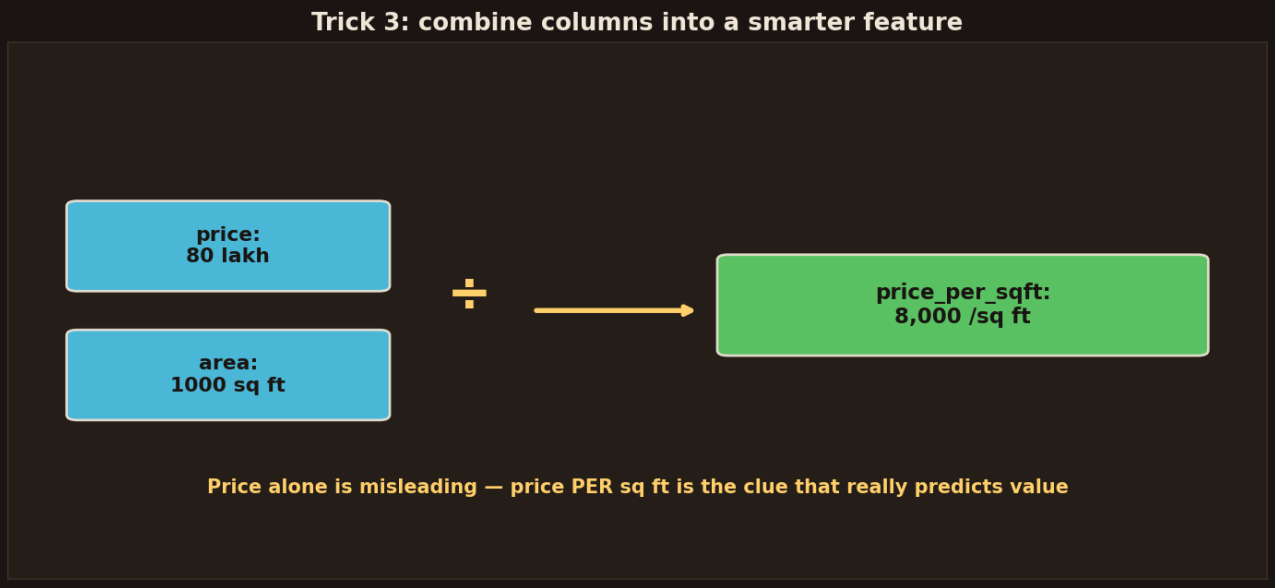
Before scaling, salary (50,000) is a giant number and age (30) is tiny — salary drowns out age. After scaling, both sit on the same 0–1 ruler, so the model treats them fairly.

Imagine two columns: **salary** (like 50,000) and **age** (like 30). To a distance-based model, salary's huge numbers **drown out** age completely — age barely counts.

- **Before:** salary's big numbers dominate; age is ignored.
- **After scaling:** squeeze every column onto the **same 0–1 ruler**, so each one gets a fair say.

> You met this warning in KNN, K-Means, SVM, and PCA — anything that measures distance. **Always scale your features** so no single big-numbered column bullies the rest. It's a one-line fix (`StandardScaler`) that quietly makes many models much better.

Trick 4: combine columns into a smarter clue



Price alone (80 lakh) is misleading — a huge house should cost more. But price ÷ area gives price_per_sqft, which truly predicts value. Combining two columns creates a smarter feature than either alone.

Sometimes the best clue is **hidden between** two columns. Take house **price** and house **area:**

- **Price alone** is misleading — of course a bigger house costs more.
- But **price ÷ area = price per sq ft** — that reveals whether a house is *actually* expensive or cheap for its size.

> This is the creative heart of feature engineering: **use your human knowledge** to combine columns into a feature the model couldn't discover on its own. Price-per-sqft, speed = distance ÷ time, orders-per-customer — these hand-built clues are often the ones that make a model click.

See it in code

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Trick 1: split a date
df["day"]        = df["date"].dt.day
df["is_weekend"] = df["date"].dt.weekday >= 5

# Trick 2: words -> numbers (one-hot)
df = pd.get_dummies(df, columns=["city"])

# Trick 4: combine columns
df["price_per_sqft"] = df["price"] / df["area"]

# Trick 3: scale everything to the same ruler
X_scaled = StandardScaler().fit_transform(X)
```

> A few lines, huge impact. `dt.day` splits dates, `get_dummies` one-hot-encodes words, a simple division makes a smart new column, and `StandardScaler` puts everything on one ruler. Master these and you've mastered 90% of everyday feature engineering.

Recap — the 20-second version

- Feature engineering = the **chef's prep** — better features beat a fancier model.
- **Split** rich columns (a date → day, month, weekday, is_weekend).
- **Encode** words into 0/1 numbers (one-hot) — models eat numbers, not words.
- **Scale** everything to the same ruler so no big column bullies the rest.
- **Combine** columns with human insight (price ÷ area = price per sq ft).

Next up — Video 20: Ensemble Methods — the wisdom of many models. You've seen one forest beat one tree. Next we learn the general trick behind almost every winning ML solution: cleverly combining many models into one super-model. See you Day 20.